

ML Handbook: Сборник материалов по теории

Метрические методы, Метрики качества и Линейные модели

11 мая 2026 г.

Содержание

1	Типология задач машинного обучения	4
2	Математический аппарат и обозначения	4
2.1	Обозначения	4
2.2	Решение МНК (Ordinary Least Squares)	5
2.3	Численные методы: QR-разложение	5
2.4	Градиентный спуск (Gradient Descent)	5
3	Обработка категориальных признаков	5
3.1	One-Hot Encoding (OHE)	6
4	Интерпретируемость моделей	6
4.1	Линейные модели	6
4.2	Решающие деревья	7
5	Свойства и ограничения линейных моделей	7
5.1	Свойства	7
5.2	Ограничения	7
6	Сведение обучения к задаче оптимизации	7
6.1	Функция потерь и Функционал качества	8
6.2	Stochastic Gradient Descent (SGD) и Mini-batches	8
7	Регуляризация и борьба с мультиколлинеарностью	8
7.1	Мультиколлинеарность	8
7.2	L2-регуляризация (Ridge Regression)	8
7.3	L1-регуляризация (Lasso)	9
7.4	Сравнительная таблица	9
8	Логистическая регрессия	9
8.1	Сигмоида (σ)	9
8.2	Метод максимального правдоподобия (MLE)	9
8.3	Изменения пространства признаков	10
9	Функции потерь классификации	10
9.1	Perceptron Loss	10
9.2	Hinge Loss и SVM	10

10 Kernel Trick и ядерные методы	11
10.1 Суть метода	11
10.2 Виды ядер	11
10.3 Kernel Density Estimation (KDE)	11
11 Многоклассовая классификация	12
11.1 Вероятности и Softmax	12
11.2 Функция потерь: Cross-Entropy	12
11.3 Алгоритмы сведения к бинарным задачам	12
12 Deep Q&A: Вопросы и подробные ответы	13
13 Типология метрик	14
13.1 Офлайновые метрики	14
13.2 Онлайновые метрики	14
13.3 Ключевые отличия	14
14 Функция потерь vs Метрика качества	14
14.1 Функция потерь (Loss Function)	15
14.2 Метрика качества (Metric)	15
14.3 В чем ключевые отличия?	15
15 Матрица ошибок (Confusion Matrix)	15
15.1 Основные метрики качества	16
16 Кривые и площади: ROC-AUC и PR-AUC	17
16.1 ROC-AUC (Receiver Operating Characteristic)	17
16.2 PR-AUC (Precision-Recall Curve)	18
17 Метрики регрессии	18
17.1 MSE (Mean Squared Error)	19
17.2 MAE (Mean Absolute Error)	19
17.3 Относительные метрики: MAPE, SMAPE, WAPE	20
17.4 RMSLE (Root Mean Squared Logarithmic Error)	20
18 Алгоритм k-ближайших соседей (k-NN)	21
18.0.1 Математическая формулировка	21
18.0.2 Классификация и Регрессия	21
18.0.3 Выбор параметров	21
19 Метрики и функции расстояния	21
20 Взвешенный k-NN	22
20.1 Веса по рангу (порядковому номеру)	22
20.2 Веса от расстояния	22
20.3 Ядерное сглаживание	22
20.4 Ядерная регрессия (Формула Надарая-Ватсона)	23
20.5 Оптимизация поиска ближайшего соседа	23
20.6 Фундаментальные свойства решающих деревьев	24
20.7 Жадный алгоритм построения	25
20.8 Критерии ветвления и информативность	25
20.9 Работа с особенностями данных	26

20.10	Регуляризация	26
20.11	Алгоритмические оптимизации (Трюки)	26
20.12	Подбор гиперпараметров: определение	28
20.13	Ключевые гиперпараметры для популярных алгоритмов	28
20.14	Методология подбора	29
20.14.1	Схема Train-Validation-Test	29
20.15	Кросс-валидация (k-Fold)	30
20.16	Grid Search (Поиск по сетке)	30
20.17	Random Search (Случайный поиск)	31
20.18	Продвинутые методы подбора	31
20.18.1	Байесовская оптимизация	31
20.18.2	Алгоритм TPE (Tree-structured Parzen Estimator)	32
20.18.3	Population Based Training (PBT)	32
20.18.4	Hyperband и Successive Halving	32
20.19	Опенсорс-библиотеки и инструменты	32
20.20	Что такое кросс-валидация?	35
20.21	Метод Hold-out	35
20.22	Стратификация (Stratification)	35
20.23	k-Fold кросс-валидация	36
20.24	Leave-one-out (LOO)	36
20.25	Stratified k-Fold	36
20.26	Кривые обучения (Learning Curves)	37
20.26.1	Как читать графики?	37
20.27	Когда стоит заподозрить, что оценка качества завышена?	37
20.28	Примеры «подмешивания» тестовых данных (Data Leakage)	38
21	Фундаментальные основы ансамблей	40
21.1	Дилемма смещения и разброса (Bias-Variance Tradeoff)	40
21.2	Случайный лес (Random Forest)	40
22	Градиентный бустинг (GBDT)	41
22.1	Философское отличие: Бустинг vs Бэггинг	41
22.2	Математика: Градиентный спуск в функциональном пространстве	41
22.2.1	Алгоритм по шагам	41
22.2.2	Почему это градиентный спуск?	42
22.2.3	Обучение на остатки	42
22.2.4	Сходимость	42
22.3	Ограничение сложности и Регуляризация	43
22.3.1	Почему деревья должны быть неглубокими?	43
22.3.2	Методы борьбы с переобучением (Регуляризация)	44
23	Тройка лидеров: XGBoost, LightGBM, CatBoost	44
23.1	XGBoost (eXtreme Gradient Boosting)	44
23.2	LightGBM (от Microsoft)	44
23.3	CatBoost (от Яндекса)	45
23.4	Интерпретация: Feature Importance	45

1 Типология задач машинного обучения

В обучении с учителем (*Supervised Learning*) мы выделяем следующие основные задачи:

- **Регрессия (Regression):** Целевая переменная $y \in \mathbb{R}$ является непрерывной. Мы ищем функциональную зависимость, которая минимизирует ошибку предсказания вещественного числа.
 - *Примеры:* Прогноз стоимости недвижимости на основе характеристик (площадь, район), предсказание температуры на завтра, оценка времени прибытия курьера (ETA).
 - *Метрики:* MSE, MAE, R^2 .
- **Бинарная классификация (Binary Classification):** $y \in \{0, 1\}$ (или $\{-1, 1\}$). Необходимо разделить объекты на две непересекающиеся группы. Модель обычно выдает вероятность $p \in [0, 1]$.
 - *Примеры:* Спам-фильтр (письмо спам или нет), кредитный скоринг (вернет ли клиент кредит), медицинская диагностика (наличие или отсутствие заболевания).
 - *Метрики:* Accuracy, Precision, Recall, F1-score, ROC-AUC.
- **Многоклассовая классификация (Multiclass Classification):** $y \in \{1, \dots, K\}$. Выбор одного из K взаимоисключающих классов.
 - *Примеры:* Распознавание рукописных цифр (0-9), классификация типов ирисов (Setosa, Versicolour, Virginica), определение языка текста.
 - *Подходы:* One-vs-Rest, One-vs-One, Softmax.
- **Ранжирование (Ranking):** Задача не в предсказании точного значения, а в восстановлении правильного **порядка** объектов.
 - *Примеры:* Выдача поисковой системы (релевантные ссылки выше), ранжирование товаров в ленте рекомендаций, сортировка кандидатов в системе подбора персонала.
 - *Метрики:* nDCG, MRR, Precision@k.

2 Математический аппарат и обозначения

2.1 Обозначения

Для формального описания линейных моделей введем следующие обозначения:

- $\mathbf{X} \in \mathbb{R}^{N \times D}$ — матрица «объекты-признаки», где N — число объектов, D — число признаков.
- $\mathbf{y} \in \mathbb{R}^N$ — вектор истинных ответов (таргетов).
- $\boldsymbol{\omega} \in \mathbb{R}^D$ — вектор параметров (весов) модели.
- Предсказание модели: $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\omega}$.

2.2 Решение МНК (Ordinary Least Squares)

Мы минимизируем сумму квадратов ошибок:

$$Q(\omega) = \|\mathbf{X}\omega - \mathbf{y}\|_2^2 = (\mathbf{X}\omega - \mathbf{y})^T(\mathbf{X}\omega - \mathbf{y})$$

Аналитическое решение находится через приравнивание градиента к нулю:

$$\nabla_{\omega} Q(\omega) = 2\mathbf{X}^T\mathbf{X}\omega - 2\mathbf{X}^T\mathbf{y} = 0$$

Откуда получаем нормальное уравнение:

$$\omega = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Условие существования

Решение существует и единственно, если матрица $\mathbf{X}^T\mathbf{X}$ невырождена (имеет полный ранг). Это эквивалентно линейной независимости признаков (отсутствию мультиколлинеарности).

2.3 Численные методы: QR-разложение

Прямое вычисление $(\mathbf{X}^T\mathbf{X})^{-1}$ вычислительно дорого $O(D^3)$ и неустойчиво. Метод **QR-разложения** позволяет решить задачу стабильнее:

1. Раскладываем $\mathbf{X} = \mathbf{Q}\mathbf{R}$, где $\mathbf{Q}$ — ортогональная матрица ($\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$), а $\mathbf{R}$ — верхнетреугольная.
2. Подставляем в нормальное уравнение: $(\mathbf{R}^T\mathbf{Q}^T\mathbf{Q}\mathbf{R})\omega = \mathbf{R}^T\mathbf{Q}^T\mathbf{y}$.
3. Упрощаем: $\mathbf{R}^T\mathbf{R}\omega = \mathbf{R}^T\mathbf{Q}^T\mathbf{y} \implies \mathbf{R}\omega = \mathbf{Q}^T\mathbf{y}$.
4. Система с треугольной матрицей $\mathbf{R}$ решается методом обратной подстановки за $O(D^2)$.

2.4 Градиентный спуск (Gradient Descent)

Если аналитическое решение невозможно (огромные данные) или функция потерь неквадратична, используется итерационный подход:

$$\omega_{t+1} = \omega_t - \eta \nabla Q(\omega_t)$$

Где η — шаг обучения (*learning rate*). Для OLS шаг градиентного спуска выглядит так:

$$\omega_{t+1} = \omega_t - 2\eta\mathbf{X}^T(\mathbf{X}\omega_t - \mathbf{y})$$

Градиент направлен в сторону наискорейшего роста функции, поэтому мы вычитаем его, чтобы двигаться к минимуму.

3 Обработка категориальных признаков

Линейные модели требуют численного представления данных. Для работы с качественными переменными (город, профессия) применяется кодирование.

3.1 One-Hot Encoding (ОНЕ)

Метод заменяет один признак с M категориями на M новых бинарных столбцов.

- **Где можно использовать:**

- Категории не имеют естественного порядка (номинальные данные).
- *Пример:* Цвета автомобиля (красный, синий, зеленый), страны, типы почтовых отправлений.
- Линейные модели и нейросети — ОНЕ позволяет модели выучить независимый вес для каждой категории.

- **Где нельзя или не рекомендуется использовать:**

- **Высокая кардинальность:** Если у признака тысячи уникальных значений (напр., ID пользователя), ОНЕ создаст разреженную матрицу огромного размера, что приведет к переобучению и нехватке памяти.
- **Деревья решений:** ОНЕ "размывает" информацию. Дереву проще работать с порядковым кодированием (*Label Encoding*), так как оно может делать сплиты по группам категорий.
- **Порядковые данные:** Если категории имеют смысл "больше-меньше" (напр., размер одежды S, M, L), лучше использовать *Ordinal Encoding*, чтобы сохранить информацию о порядке.

Dummy Variable Trap

Для линейных моделей с интерсептом (w_0) необходимо удалять один столбец после ОНЕ ($M - 1$ признаков). В противном случае возникает строгая мультиколлинеарность, так как сумма всех ОНЕ-столбцов всегда равна единице.

4 Интерпретируемость моделей

Это способность модели объяснить, на основе каких признаков и почему было принято решение. По уровню прозрачности алгоритмы делятся на *White-box* (прозрачные) и *Black-box* (непрозрачные).

4.1 Линейные модели

Линейные модели относятся к наиболее интерпретируемым алгоритмам.

- **Веса ω_i :** Показывают вклад признака x_i . Если $\omega_i > 0$, рост признака увеличивает предсказание. Если $\omega_i < 0$ — уменьшает.
- **Абсолютное значение $|\omega_i|$:** Если признаки стандартизированы, то чем больше модуль веса, тем важнее признак для модели (*Feature Importance*).
- **Ограничение:** При наличии мультиколлинеарности веса теряют физический смысл и становятся неинтерпретируемыми.

4.2 Решающие деревья

Деревья интерпретируются через их структуру и правила принятия решений.

- **Путь от корня к листу:** Представляет собой цепочку логических условий (IF-THEN), понятных человеку без специальной подготовки.
- **Feature Importance:** Рассчитывается на основе того, насколько сильно каждый признак уменьшает неопределенность (энтропию или критерий Джини) во всех узлах дерева.
- *Ограничение:* Глубокие деревья («переобученные») становятся слишком сложными для визуального анализа человеком.

5 Свойства и ограничения линейных моделей

Понимание сильных и слабых сторон линейных алгоритмов необходимо для правильного выбора модели и качественной подготовки признаков.

5.1 Свойства

- **Простота и скорость:** Обучение и предсказание выполняются крайне быстро, что критично для высоконагруженных систем.
- **Интерпретируемость:** Прямая связь между признаками и целевой переменной через веса.
- **Работа с редкими данными:** Хорошо справляются с разреженными матрицами (sparse data) после ONE.

5.2 Ограничения

- **Линейность зависимости:** Модель не может выучить нелинейные паттерны и взаимодействия признаков (напр., $x_1 \cdot x_2$) без их явного добавления вручную (*Polynomial Features*).
- **Чувствительность к выбросам:** Квадратичная ошибка в МНК придает аномалиям слишком большой вес, что может "развернуть" разделяющую плоскость. Решение — использование Huber Loss или MAE.
- **Мультиколлинеарность:** При наличии сильно коррелирующих признаков веса становятся огромными и неустойчивыми. Требуется регуляризация.
- **Масштаб признаков:** Веса напрямую зависят от единиц измерения. Для корректной работы регуляризации и интерпретации обязательна нормализация.

6 Сведение обучения к задаче оптимизации

Процесс «обучения» модели в ML формализуется как поиск параметров, минимизирующих некоторую функцию ошибки на обучающей выборке.

6.1 Функция потерь и Функционал качества

Важно различать ошибку на одном объекте и на всей выборке:

- **Функция потерь** $L(a(x), y)$: Определяет величину ошибки на *одном* конкретном объекте x .
 - Для регрессии: $(a(x) - y)^2$ (квадратичная), $|a(x) - y|$ (абсолютная).
 - Для классификации: $\log(1 + \exp(-y \cdot a(x)))$ (логистическая).
- **Функционал качества** $Q(w)$: Суммарная ошибка на всей обучающей выборке X :

$$Q(w) = \frac{1}{N} \sum_{i=1}^N L(a(x_i, w), y_i)$$

Задача обучения — найти $w^* = \arg \min_w Q(w)$.

6.2 Stochastic Gradient Descent (SGD) и Mini-batches

Когда данных слишком много для вычисления градиента по всей выборке (Full Batch), используются стохастические методы:

1. **SGD**: На каждом шаге выбирается один случайный объект i , и веса обновляются по его градиенту: $w \leftarrow w - \eta \nabla L(x_i)$. Это очень быстро, но траектория обучения сильно "шумит".
2. **Mini-batch GD**: Золотая середина. Мы выбираем небольшое подмножество данных (*batch*) размера B (напр., 32, 64, 128) и вычисляем средний градиент по ним:

$$\nabla Q_{batch} = \frac{1}{B} \sum_{j=1}^B \nabla L(x_j)$$

Это позволяет эффективно использовать векторизацию вычислений на CPU/GPU и уменьшить шум по сравнению с чистым SGD.

7 Регуляризация и борьба с мультиколлинеарностью

7.1 Мультиколлинеарность

Это ситуация, когда признаки в матрице $\mathbf{X}$ сильно коррелируют друг с другом.

- **Последствия**: Матрица $\mathbf{X}^T \mathbf{X}$ становится близкой к вырожденной. Это приводит к неустойчивости оценок весов: малейшее изменение в данных вызывает гигантские скачки значений ω_i .
- **Решение**: Добавление штрафа за величину весов в функционал качества.

7.2 L2-регуляризация (Ridge Regression)

Добавляем квадрат $L2$ -нормы весов: $Q_{reg}(\omega) = Q(\omega) + \lambda \|\omega\|_2^2$.

- **Аналитическое решение**: $\omega = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$.
- **Плюсы**: Всегда делает матрицу обратимой, уменьшает дисперсию весов, имеет аналитическое решение.
- **Минусы**: Не зануляет веса (не делает отбор признаков).

7.3 L1-регуляризация (Lasso)

Добавляем $L1$ -норму весов: $Q_{reg}(\omega) = Q(\omega) + \lambda \|\omega\|_1$.

- **Особенность:** Из-за "острого" края ромба (линии уровня $L1$) точка оптимума часто попадает на оси координат.
- **Плюсы: Отбор признаков** (*Feature Selection*) — зануляет веса у неважных признаков.
- **Минусы:** Нет аналитического решения, нестабильна при сильной корреляции (выбирает один случайный признак из группы коррелирующих).

7.4 Сравнительная таблица

Свойство	L2 (Ridge)	L1 (Lasso)
Штраф	$\sum \omega_i^2$	$\sum \omega_i $
Решение	Аналитическое	Численное
Отбор признаков	Нет (веса малы, но $\neq 0$)	Да (зануляет лишнее)
Устойчивость	Высокая	Низкая (чувствительна к шуму)
Интерпретация	Сохраняет все признаки	Дает разреженную модель

8 Логистическая регрессия

8.1 Сигмоида (σ)

Для перевода выхода линейной модели $\omega^T x \in \mathbb{R}$ в вероятность $p \in [0, 1]$ используется логистическая функция (сигмоида):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Свойство:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, что удобно при расчете градиентов.
- **Смысл:** Она "сплюсчивает" бесконечную прямую в интервал $(0, 1)$, позволяя интерпретировать результат как уверенность модели.

8.2 Метод максимального правдоподобия (MLE)

Мы хотим подобрать такие ω , при которых наблюдаемые данные наиболее вероятны. Предположим, что $y_i \sim \text{Bernoulli}(p_i)$. Тогда правдоподобие всей выборки:

$$L(\omega) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$$

Логарифмируя и меняя знак, переходим к минимизации **Log-Loss**:

$$Q(\omega) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln(\sigma(\omega^T x_i)) + (1 - y_i) \ln(1 - \sigma(\omega^T x_i))]$$

8.3 Изменения пространства признаков

Линейные модели строят линейную разделяющую поверхность в текущем пространстве признаков. Чтобы выучить нелинейные зависимости, мы можем трансформировать исходное пространство:

- **Полиномиальные признаки:** Добавление степеней и произведений признаков ($x_1, x_2 \rightarrow x_1^2, x_1x_2, x_2^2$). Это позволяет модели строить криволинейные границы (эллипсы, параболы).
- **RBF и Ядра:** Переход в бесконечномерное пространство, где данные становятся линейно separable (основа SVM с ядрами).
- **Feature Engineering:** Ручное создание признаков (напр., $\sin(x)$, $\text{sign}(x)$) на основе знаний о предметной области.

9 Функции потерь классификации

Для задач классификации с метками $y \in \{-1, 1\}$ часто используют функции от *отступа* (margin) $M = y \cdot (\omega^T x)$.

9.1 Perceptron Loss

$$L(M) = \max(0, -M)$$

- Ошибкой считаются только те объекты, которые лежат на «чужой» стороне разделяющей плоскости.
- Не дает никакой уверенности в классификации — объект может быть очень близко к границе.

9.2 Hinge Loss и SVM

$$L(M) = \max(0, 1 - M)$$

Это стандартная функция потерь для **SVM (Метод опорных векторов)**.

- **Зазор (Margin):** В отличие от перцептрона, Hinge Loss требует, чтобы объекты не просто были классифицированы верно ($M > 0$), но и находились на расстоянии хотя бы 1 от разделяющей прямой ($M \geq 1$).
- Все объекты с $M < 1$ вносят вклад в ошибку. Это обеспечивает «максимальный разделяющий зазор», что делает модель более устойчивой к шуму.

10 Kernel Trick и ядерные методы

Ядерный переход (*Kernel Trick*) позволяет линейным моделям работать в нелинейных пространствах без явного вычисления координат новых признаков.

10.1 Суть метода

Многие алгоритмы (SVM, PCA, Линейная регрессия) зависят только от скалярных произведений объектов $\langle x_i, x_j \rangle$. Мы можем заменить их на **ядерную функцию** $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$, которая вычисляет скалярное произведение в некотором пространстве более высокой размерности $\phi(\cdot)$.

10.2 Виды ядер

- **Линейное (Linear):** $K(x, y) = x^T y$. Обычное скалярное произведение. Используется, когда данных много или они уже линейно разделимы.
- **Полиномиальное (Polynomial):** $K(x, y) = (x^T y + c)^d$. Позволяет учитывать взаимодействия признаков до степени d .
- **RBF / Гауссово (Radial Basis Function):** $K(x, y) = \exp(-\gamma \|x - y\|^2)$. Бесконечномерное ядро. Самое популярное для нелинейных задач. γ определяет "радиус влияния" каждого объекта.
- **Сигмоидальное (Sigmoid):** $K(x, y) = \tanh(\alpha x^T y + c)$. Позволяет имитировать поведение двухслойной нейронной сети.

10.3 Kernel Density Estimation (KDE)

Ядерная оценка плотности — это непараметрический способ оценки функции распределения случайной величины.

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

- Вместо грубых "ящиков" гистограммы мы ставим "колокол" ядра (обычно Гауссова) над каждой точкой данных.
- Параметр *bandwidth* (h) контролирует гладкость: слишком малое h дает шум, слишком большое — переглаженное распределение.

11 Многоклассовая классификация

Когда число классов $K > 2$, бинарная логистическая регрессия обобщается до многоклассовой (Multinomial Logistic Regression).

11.1 Вероятности и Softmax

Вместо одного числа модель выдает вектор "сырых" ответов (*logits*) $z = (z_1, \dots, z_K)$. Чтобы превратить их в вероятности, используется функция **Softmax**:

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

- Сумма всех вероятностей всегда равна 1.
- Каждый компонент $P \in (0, 1)$.
- Softmax является многомерным обобщением сигмоиды.

11.2 Функция потерь: Cross-Entropy

Для многоклассовой классификации минимизируется кросс-энтропия между истинным распределением (One-Hot вектор y) и предсказанным (p):

$$Q(\omega) = - \sum_{i=1}^n \sum_{k=1}^K y_{ik} \ln p_{ik}$$

Если объект относится к классу k , то $y_{ik} = 1$, и мы просто минимизируем $-\ln p_{ik}$ (максимизируем вероятность правильного класса).

11.3 Алгоритмы сведения к бинарным задачам

Если мы не хотим использовать Softmax напрямую, можно применить стратегии:

- **One-vs-Rest (OvR)**: Обучаем K классификаторов. Каждый учится отличать свой класс от всех остальных вместе взятых. Предсказание — класс с максимальной уверенностью.
- **One-vs-One (OvO)**: Обучаем $K(K-1)/2$ классификаторов для каждой пары классов. Итоговое решение принимается голосованием большинства. Более устойчива к дисбалансу, но требует много моделей.

12 Деер Q&A: Вопросы и подробные ответы

Q1: Почему в Ridge-регрессии веса никогда не становятся ровно нулями?

Ответ: Линии уровня L2-штрафа — это сферы. Касание линий уровня функции потерь и сферы может произойти в любой точке. Вероятность того, что точка касания попадет точно на координатную ось в высокоразмерном пространстве, практически нулевая. В отличие от L1 (ромб), у L2 нет "углов" на осях.

Q2: Как мультиколлинеарность влияет на дисперсию весов?

Ответ: Дисперсия весов обратно пропорциональна определителю $\mathbf{X}^T \mathbf{X}$. При мультиколлинеарности матрица близка к вырожденной, её определитель мал, что раздувает дисперсию оценок весов. Это делает модель крайне чувствительной к малейшим изменениям в обучающей выборке.

Q3: В чем разница между моделями One-vs-Rest и One-vs-One?

Ответ:

- **OvR:** Обучаем K классификаторов ("Класс i против всех остальных"). Быстрее, но может возникнуть дисбаланс классов.
- **OvO:** Обучаем $K(K-1)/2$ классификаторов для каждой пары. Более устойчива к дисбалансу, но вычислительно дороже при большом числе классов.

Q4: Зачем нужна калибровка вероятностей (напр. по Платту)?

Ответ: Некоторые модели (например, SVM) не выдают вероятности напрямую. Сигмоидальная калибровка Платта обучает логистическую регрессию над выходами модели, чтобы превратить их в честные доверительные интервалы $[0, 1]$.

Q5: Что такое Rank Deficiency и как его лечить?

Ответ: Это ситуация, когда число признаков D больше числа объектов N , либо признаки линейно зависимы. В этом случае $\mathbf{X}^T \mathbf{X}$ необратима. Лечится добавлением L2-регуляризации, которая делает матрицу $(\mathbf{X}^T \mathbf{X} + \lambda I)$ строго положительно определенной и всегда обратимой.

Antigravity: Этот уровень проработки теории отличает Senior ML-инженера от новичка. Помните: любая сложная нейросеть — это всего лишь композиция простых линейных преобразований и нелинейностей.

13 Типология метрик

13.1 Офлайнные метрики

Офлайнные метрики вычисляются на исторических данных (тестовых выборках) без участия реальных пользователей.

- **Зачем нужны:**
 - Для быстрой итерации и сравнения моделей между собой.
 - Для отбора лучших кандидатов перед запуском дорогостоящих А/В-тестов.
 - Для отладки алгоритма и поиска его слабых мест (анализ ошибок).
- **Примеры:** Accuracy, Precision, Recall, ROC-AUC, RMSE, MAE.

13.2 Онлайнные метрики

Онлайнные метрики измеряются в реальном времени при взаимодействии системы с «живыми» пользователями в ходе А/В-тестирования.

- **Зачем нужны:**
 - Для оценки реального влияния модели на бизнес-показатели.
 - Для учета поведенческих факторов и внешних эффектов (например, сезонность или действия конкурентов).
 - Для проверки «прокси-гипотез» (коррелирует ли рост ROC-AUC с ростом выручки).
- **Примеры:** CTR (Click-Through Rate), Конверсия (CR), ARPU (Average Revenue Per User), время сессии, Retention.

13.3 Ключевые отличия

Характеристика	Офлайн	Онлайн
Источник данных	Исторические данные	Реальные пользователи
Скорость получения	Минуты/часы	Дни/недели
Стоимость	Дешево (CPU/GPU)	Дорого (риск потери прибыли)
Объективность	Оценивает модель	Оценивает продукт/бизнес

Главная проблема ML в продакшене — отсутствие корреляции между офлайн и онлайн метриками. Модель может идеально предсказывать вероятность клика (высокий ROC-AUC), но показывать плохой CTR из-за того, что предлагаемые товары неинтересны пользователям в данный момент.

14 Функция потерь vs Метрика качества

Часто новички путают эти понятия, так как в некоторых задачах (например, в регрессии) они могут совпадать (MSE). Однако это концептуально разные вещи.

14.1 Функция потерь (Loss Function)

Это математический инструмент для **обучения** модели.

- **За что отвечает:** Показывает, насколько сильно модель «ошиблась» на конкретном объекте или выборке.
- **Как используется:** Минимизируется градиентными методами (SGD, Adam) для настройки весов.
- **Требования:** Должна быть **дифференцируемой** (или иметь субградиент), чтобы мы могли вычислить производную.
- **Примеры:** LogLoss, Cross-Entropy, Huber Loss.

14.2 Метрика качества (Metric)

Это инструмент для **оценки** модели человеком или бизнесом.

- **За что отвечает:** Показывает реальное качество работы модели в понятных единицах (проценты, рубли, количество ошибок).
- **Как используется:** Для принятия решения о запуске модели, сравнения алгоритмов, мониторинга.
- **Требования:** Должна быть **интерпретируемой**. Не обязана быть дифференцируемой.
- **Примеры:** Accuracy, F1-score, ROC-AUC.

14.3 В чем ключевые отличия?

1. **Цель:** Loss — для оптимизатора, Metric — для человека.
2. **Гладкость:** Функция потерь должна быть гладкой. Метрика может быть дискретной (например, Accuracy меняется скачками).
3. **Интерпретируемость:** Мы редко можем сказать, что означает «LogLoss = 0.23» в терминах бизнеса, но «Accuracy = 95%» понятно всем.

Почему нельзя просто минимизировать Accuracy? Потому что её производная везде равна нулю (или не существует), и градиентный спуск не сможет сдвинуть веса модели с места. Поэтому мы используем LogLoss как «дифференцируемый суррогат» для Accuracy.

15 Матрица ошибок (Confusion Matrix)

Матрица ошибок — это таблица, позволяющая визуализировать производительность алгоритма классификации.

	Y=1 (Positive)	Y=0 (Negative)
a(x)=1	TP (True Positive)	FP (False Positive)
a(x)=0	FN (False Negative)	TN (True Negative)

Таблица 2: Матрица ошибок для бинарной классификации.

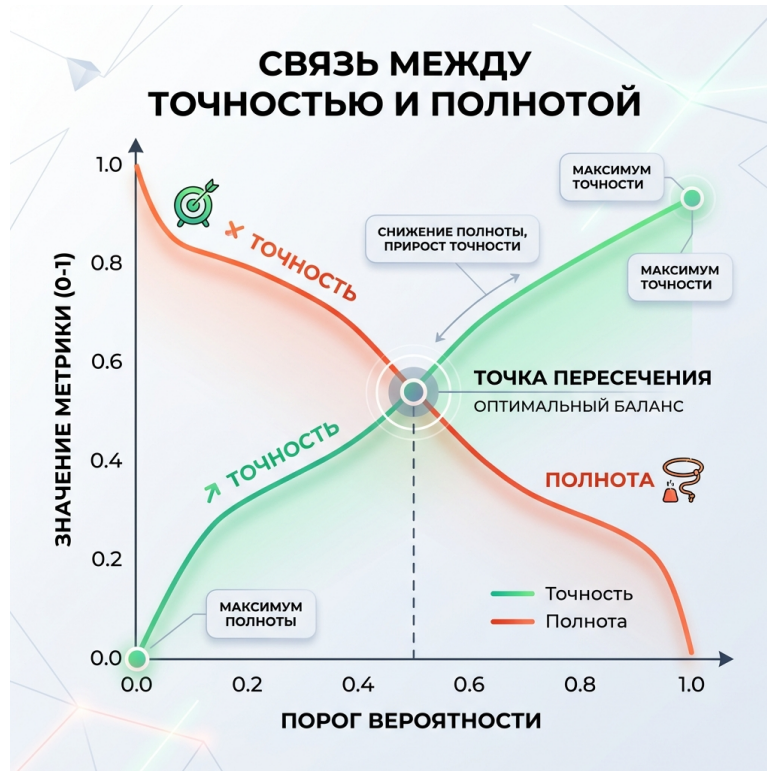


Рис. 1: Компромисс между точностью и полнотой.

15.1 Основные метрики качества

На основе матрицы ошибок вычисляются базовые метрики:

- **Accuracy** (Доля правильных ответов): Показывает общую точность модели.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision** (Точность): Доля истинно положительных среди всех объектов, которые алгоритм отнес к положительному классу. Помогает минимизировать количество ложных срабатываний (FP).

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** (Полнота): Доля истинно положительных среди всех объектов положительного класса. Помогает минимизировать количество пропусков (FN).

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-measure** (F-мера): Гармоническое среднее между точностью и полнотой. Позволяет оценить модель одним числом, когда важны и Precision, и Recall.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Precision и Recall часто находятся в противоречии: пытаясь улучшить одну метрику, мы обычно ухудшаем другую. Выбор баланса между ними зависит от цены ошибки (что хуже: ложная тревога или пропуск цели?).

Метрика	Пример задачи	Где ЗАПРЕЩЕНО
Accuracy	Распознавание цифр	Фрод (сильный дисбаланс)
Precision	Таргетинг (высокая цена звонка)	Поиск пропавших (важен каждый сигнал)
Recall	Диагностика рака (нельзя пропустить)	Спам-фильтр (важные письма не в спам)
F1-measure	Поиск в базе знаний	Когда Recall критичнее Precision

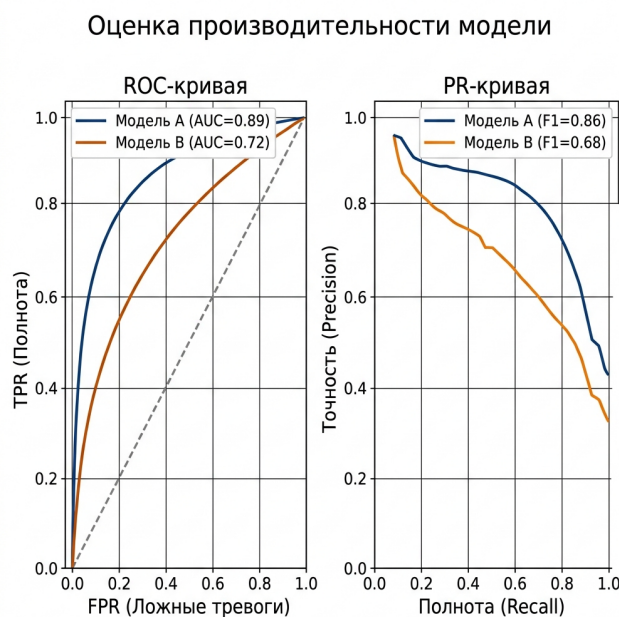


Рис. 2: Сравнение ROC и PR кривых.

16 Кривые и площади: ROC-AUC и PR-AUC

Эти метрики оценивают качество ****ранжирования**** объектов моделью, а не просто точность предсказания классов при фиксированном пороге.

16.1 ROC-AUC (Receiver Operating Characteristic)

Кривая строится в осях:

- **TPR** (True Positive Rate / Recall): $\frac{TP}{TP+FN}$.

- **FPR** (False Positive Rate): $\frac{FP}{FP+TN}$.

Суть: Показывает долю пар объектов разного класса, которые алгоритм верно упорядочил (положительный объект имеет оценку выше отрицательного).

- **Плюсы:**

- Не зависит от порога классификации.
- Устойчива к изменению баланса классов (формулы TPR и FPR нормируются внутри своего класса).

- **Минусы:**

- На сильно несбалансированных данных (например, 1:1000) может быть обманчиво высокой, даже если модель ошибается на большинстве реальных позитивных объектов.

- **Где подходит:** Задачи с более-менее сбалансированными классами.

16.2 PR-AUC (Precision-Recall Curve)

Кривая строится в осях:

- **Recall** (Полнота).

- **Precision** (Точность).

- **Плюсы:**

- Явно фокусируется на качестве предсказания редкого (положительного) класса.
- Очень чувствительна к ложным срабатываниям (FP), что критично для задач с дисбалансом.

- **Минусы:**

- Сильно зависит от баланса классов в обучающей выборке. При изменении доли позитивных объектов кривая «просядет» или поднимется.

- **Где подходит:** Задачи с сильным дисбалансом (фрод, дефектоскопия, редкие заболевания).

Главное правило: если классы сбалансированы — смотри на ROC-AUC. Если есть сильный дисбаланс — PR-AUC даст гораздо более честную картину.

17 Метрики регрессии

В задачах регрессии мы предсказываем вещественное число $\hat{y} \in \mathbb{R}$.

17.1 MSE (Mean Squared Error)

Среднеквадратичная ошибка.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Плюсы:** Хорошая «математика» (выпуклая, дифференцируемая), сильно штрафует за большие отклонения.
- **Минусы:** Неустойчива к выбросам (один выброс может «утянуть» значение метрики), трудно интерпретировать (единицы измерения в квадрате).

17.2 MAE (Mean Absolute Error)

Средняя абсолютная ошибка.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Плюсы:** Устойчива к выбросам (робастна), легко интерпретируется (единицы измерения совпадают с таргетом).
- **Минусы:** Не дифференцируема в нуле (сложнее использовать как Loss).

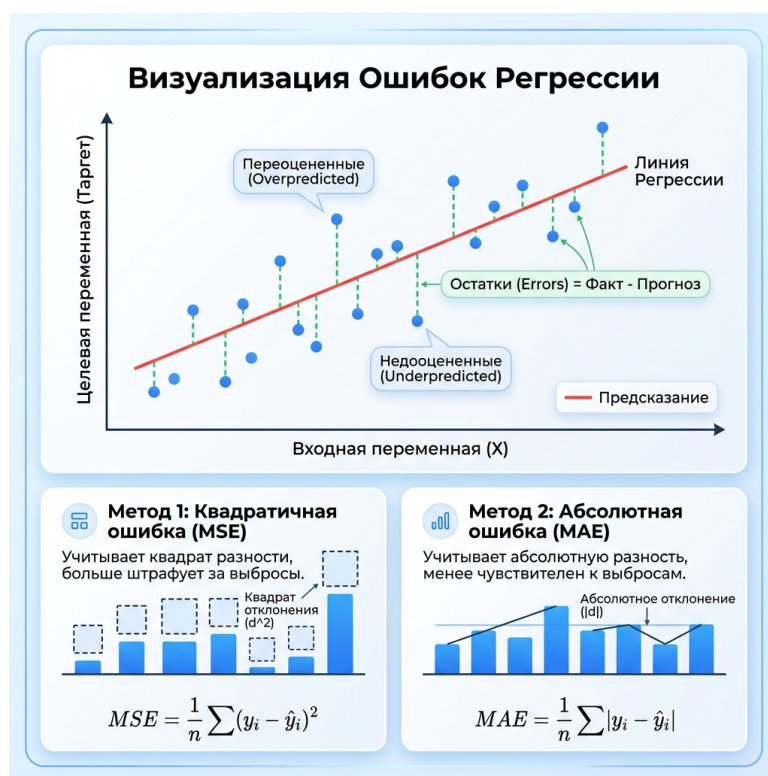


Рис. 3: Визуализация: MSE (слева) гораздо сильнее реагирует на выбросы, чем MAE (справа).

17.3 Относительные метрики: MAPE, SMAPE, WAPE

Используются, когда нам важна не абсолютная ошибка, а ошибка в процентах от величины таргета.

- **MAPE** (Mean Absolute Percentage Error):

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Плохо: нельзя использовать, если в данных есть нули ($y_i = 0$). Асимметрична: сильнее штрафует за занижение прогноза, чем за завышение.

- **SMAPE** (Symmetric MAPE):

$$SMAPE = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}$$

Решает проблему деления на ноль (если $\hat{y}_i$ не ноль) и уменьшает асимметрию.

- **WAPE** (Weighted MAPE):

$$WAPE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|}$$

Позволяет избежать проблем с околонулевыми значениями отдельных объектов, суммируя ошибки по всей выборке.

17.4 RMSLE (Root Mean Squared Logarithmic Error)

$$RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(y_i + 1) - \log(\hat{y}_i + 1))^2}$$

- **Зачем:** Когда таргет меняется на порядки (цены на жилье, продажи). Штрафует за относительную ошибку, а не абсолютную.
- **Особенность:** Сильнее штрафует за ****занижение**** прогноза ($y > \hat{y}$), чем за завышение.

Antigravity: Метрики — это мост между бизнесом и математикой. Неверный выбор метрики ведет к созданию идеальной, но бесполезной модели.

18 Алгоритм k-ближайших соседей (k-NN)

18.0.1 Математическая формулировка

Пусть задана обучающая выборка $X^\ell = \{(x_1, y_1), \dots, (x_\ell, y_\ell)\}$, где $x_i \in \mathbb{R}^d$ — векторы признаков, а y_i — целевые значения. Для произвольного объекта x определим порядок близости объектов обучающей выборки:

$$\rho(x, x_{(1)}) \leq \rho(x, x_{(2)}) \leq \dots \leq \rho(x, x_{(\ell)})$$

где $\rho(x, z)$ — функция расстояния (метрика), а $x_{(i)}$ — i -й ближайший сосед объекта x .

Важное условие: Использование метрик требует обязательной нормировки признаков, так как признаки с большими абсолютными значениями будут доминировать в расчете расстояния.

18.0.2 Классификация и Регрессия

- **Классификация:** Ответ — наиболее часто встречающийся класс среди k соседей (мода). При взвешенном голосовании: $a(x) = \operatorname{argmax}_{y \in Y} \sum_{i=1}^k w_i [y_{(i)} = y]$.
- **Регрессия:** Ответ — среднее арифметическое ответов соседей: $a(x) = \frac{1}{k} \sum_{i=1}^k y_{(i)}$. Во взвешенном варианте: $a(x) = \frac{\sum w_i y_{(i)}}{\sum w_i}$.

18.0.3 Выбор параметров

- **Число соседей k :**
 - При малых k модель имеет **низкое смещение (bias)**, но **высокий разброс (variance)**. Риск переобучения.
 - При больших k модель имеет **высокое смещение**, но **низкий разброс**. Риск недообучения.
- **Методы выбора:** Обычно k подбирают на кросс-валидации или методом Leave-One-Out. Также используют «метод локтя» (анализ графика ошибки).

Выбор k — это классический поиск компромисса между недообучением и переобучением. Обычно k подбирают на кросс-валидации.

19 Метрики и функции расстояния

- **Семейство Минковского (L_p):** Общая формула: $\rho(x, z) = \left(\sum_{i=1}^d |x_i - z_i|^p \right)^{1/p}$.
 - L_1 (**Manhattan**): $\sum_{i=1}^d |x_i - z_i|$. Менее чувствительна к выбросам, часто лучше работает в пространствах высокой размерности.
 - L_2 (**Euclidean**): $\sqrt{\sum_{i=1}^d (x_i - z_i)^2}$. Стандартное расстояние «по прямой». Чувствительно к большим отклонениям в отдельных признаках.
 - L_∞ (**Chebyshev**): $\max_i |x_i - z_i|$. Расстояние определяется максимальным различием по одной из координат.

- **Косинусное расстояние:** $d_{\cos}(x, z) = 1 - \frac{\langle x, z \rangle}{\|x\| \|z\|} = 1 - \frac{\sum x_i z_i}{\sqrt{\sum x_i^2} \sqrt{\sum z_i^2}}$. Измеряет угол между векторами, игнорируя их норму (длину). Широко применяется в анализе текстов и NLP.
- **Расстояние Жаккара:** $d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$. Используется для сравнения множеств или бинарных признаков (например, наличие слов в документе). Отражает долю общих элементов от их общего количества.

*Помните, что перед использованием L_p метрик данные **обязательно** нужно масштабировать, иначе признак с большим диапазоном значений «поглотит» все остальные.*

20 Взвешенный k-NN

20.1 Веса по рангу (порядковому номеру)

Вес соседа зависит не от расстояния до него, а от его места в списке ближайших соседей.

- **Линейно убывающие веса:**

$$w_i = \frac{k + 1 - i}{k}$$

- **Экспоненциально убывающие веса:**

$$w_i = q^i, \quad 0 < q < 1$$

20.2 Веса от расстояния

Вес вычисляется непосредственно как функция от расстояния $\rho(x, x_{(i)})$.

- **Обратное расстояние:**

$$w_i = \frac{1}{\rho(x, x_{(i)}) + \epsilon}$$

- **Относительное расстояние:**

$$w_i = \frac{\rho(x, x_{(k+1)}) - \rho(x, x_{(i)})}{\rho(x, x_{(k+1)}) - \rho(x, x_{(1)})}$$

20.3 Ядерное сглаживание

Вес соседа определяется через функцию ядра $K(z)$:

$$w_i = K\left(\frac{\rho(x, x_i)}{h}\right)$$

- **Типы ядер:**

- **Епанечникова:**

$$K(z) = \frac{3}{4}(1 - z^2)[|z| \leq 1]$$

- **Квартическое:**

$$K(z) = \frac{15}{16}(1 - z^2)^2[|z| \leq 1]$$

- **Гауссовское:**

$$K(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}$$

20.4 Ядерная регрессия (Формула Надарая-Ватсона)

Это обобщение k-NN для задачи регрессии, где предсказание — это взвешенное среднее ответов всех объектов обучающей выборки:

$$a(x) = \frac{\sum_{i=1}^n y_i K\left(\frac{\rho(x, x_i)}{h}\right)}{\sum_{i=1}^n K\left(\frac{\rho(x, x_i)}{h}\right)}$$

20.5 Оптимизация поиска ближайшего соседа

Для работы с большими данными ($N \gg 1$) используются методы, позволяющие избежать полного перебора объектов.

- **Точные методы:**

- **k-d деревья:** Разбивают пространство по признакам. Эффективны только при малых d (< 20). При высокой размерности деградируют до $O(N)$.
- **Ball trees:** Разбивают пространство на гиперболы. Работают стабильнее k-d деревьев в пространствах средней размерности.

- **Приближенные методы (ANN):**

- **LSH (Locality-Sensitive Hashing):** Хеширование, при котором близкие объекты с высокой вероятностью попадают в один бакет. Поиск идет только внутри бакета.
- **HNSW (Hierarchical Navigable Small World):** Построение графа, где вершины — объекты, а ребра соединяют соседей. Иерархическая структура позволяет быстро выходить в нужную область пространства и уточнять результат.
- **Inverted File Index (IVF):** Пространство делится на кластеры (ячейки Ворового), и поиск ведется только в ближайших к запросу кластерах.

Antigravity: В реальном продакшене (Faiss, HNSW) почти всегда используются именно ANN, так как точный поиск на миллионах объектов невозможен.

20.6 Фундаментальные свойства решающих деревьев

- **Вопрос:** Почему функция дерева является кусочно-постоянной и как это влияет на оптимизацию?
- **Ответ:** Решающее дерево представляет собой модель, которая разбивает признаковое пространство на J непересекающихся гиперпрямоугольников (листов) $L_1, \dots, L_J$. Математически предсказание записывается как:

$$a(x) = \sum_{j=1}^J c_j [x \in L_j], \text{ где } c_j \text{ — константа в листе.}$$

Так как индикаторная функция $[x \in L_j]$ постоянна внутри области и имеет разрывы на границах, производная $a(x)$ по параметрам разбиения (порогам t) почти везде равна нулю. Это делает **градиентную оптимизацию (SGD) невозможной** для поиска оптимальной структуры (сплитов). Вместо этого используются жадные алгоритмы дискретной оптимизации.

- **Вопрос:** Почему дерево не умеет экстраполировать?
- **Ответ:** В отличие от линейной регрессии, которая может предсказывать сколь угодно большие значения, дерево «заперто» в диапазоне значений обучающей выборки. Любой объект x , находящийся далеко за пределами распределения признаков обучения, неизбежно попадет в один из существующих «крайних» листов. Таким образом, предсказанное значение всегда будет лежать в интервале $[\min(y_{train}), \max(y_{train})]$, и модель никогда не предскажет «новый рекорд», которого не было в данных.
- **Вопрос:** Как задается предикат в узлах дерева?
- **Ответ:** Предикат — это бинарное решающее правило в каждой внутренней вершине. В большинстве реализаций (CART, ID3) используются **осепараллельные** (axis-aligned) предикаты вида:

$$B(x, j, t) = [x_j \leq t]$$

где x_j — значение j -го признака объекта x , а $t \in \mathbb{R}$ — порог. Если условие выполняется, объект уходит в левое поддерево, иначе — в правое. Использование именно таких простых предикатов позволяет эффективно перебирать пороги, но заставляет дерево аппроксимировать диагональные зависимости «ступеньками».

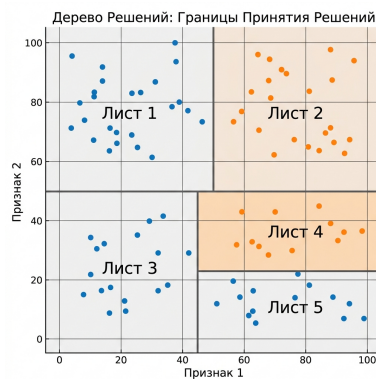


Рис. 4: Разбиение признакового пространства на гиперпрямоугольники (листы).

Дерево — это отличный интерполятор, но плохой предсказатель «будущих рекордов». Оно никогда не предскажет цену квартиры выше, чем та, что была в обучении.

20.7 Жадный алгоритм построения

- **Вопрос:** Почему построение оптимального дерева — NP-трудная задача и в чем суть реализации «жадного» подхода?
- **Ответ:** Поиск глобально оптимального дерева требует перебора всех возможных структур и последовательностей разбиений. Количество таких вариантов растет как сверхэкспонента от числа признаков и объектов.
- **Реализация (ID3, C4.5, CART):** Используется алгоритм **рекурсивного разбиения** (Recursive Partitioning):
 1. Начинаем с корневого узла, содержащего всю обучающую выборку X .
 2. В текущем узле перебираем все возможные пары (признак j , порог t).
 3. Для каждой пары вычисляем *критерий информативности* (индикация того, насколько «чище» станут данные после сплита).
 4. Выбираем пару (j, t) , дающую максимальный выигрыш (**жадный выбор**).
 5. Разбиваем выборку на две части и рекурсивно повторяем шаги для дочерних узлов до срабатывания критериев остановки (остановка рекурсии).

Жадность — это как выбор самого вкусного куса торта прямо сейчас вместо того, чтобы сесть на диету ради здоровья в будущем. Это работает быстро, но не гарантирует идеальный результат для всей структуры.

20.8 Критерии ветвления и информативность

- **Gain (Прирост информативности):** $G(Q, j, t) = I(Q) - \frac{|Q_L|}{|Q|} I(Q_L) - \frac{|Q_R|}{|Q|} I(Q_R)$, где Q — выборка в узле, I — функция информативности. Мы максимизируем G , минимизируя взвешенную неопределенность в детях.
- **Классификация:**
 - **Энтропия Шеннона:** $H(p) = -\sum p_k \log p_k$. Основана на теории информации. Более «чувствительна» к изменениям вероятностей классов, чем Джини, и стремится создавать максимально сбалансированные или идеально чистые узлы.
 - **Критерий Джини:** $G = \sum p_k(1 - p_k) = 1 - \sum p_k^2$. Математически это вероятность ошибки при случайном выборе класса из распределения в узле. Джини вычислительно эффективнее (нет логарифмов) и чаще используется по умолчанию (например, в sklearn).
- **Регрессия:**
 - **MSE (Mean Squared Error):** $I(Q) = \frac{1}{|Q|} \sum (y_i - \bar{y})^2$. Это дисперсия ответов. Минимизация MSE заставляет модель предсказывать среднее значение по листу. *Почему хорошо?* Она математически удобна и сильно штрафует за большие отклонения.
 - **MAE (Mean Absolute Error):** $I(Q) = \frac{1}{|Q|} \sum |y_i - \text{med}(y)|$. Предсказывает медиану. *Почему хорошо?* Она более устойчива (робастна) к выбросам в данных, чем MSE.

Критерий доли ошибок плох тем, что он «грубый» — он может не измениться после сплита, даже если классы стали более разделенными.

20.9 Работа с особенностями данных

- Категориальные признаки:

- **One-Hot Encoding**: Подходит для признаков с малой мощностью. Минус: сильно раздувает признаковое пространство и заставляет дерево делать много «неэффективных» сплитов.
- **Группировка по таргету (Метод CART)**: Категории сортируются по среднему значению целевой переменной (в случае регрессии или бинарной классификации). Это превращает категориальный признак в порядковый, и задача сводится к поиску порога t . Доказано, что для бинарной классификации и регрессии такой жадный поиск дает оптимальное разбиение среди всех возможных разбиений категорий на две группы.

- Пропущенные значения (Missing Values):

- **Направление по умолчанию**: Объект с пропуском отправляется в ту сторону, где ошибка на обучающей выборке была меньше, либо в сторону большинства объектов.
- **Суррогатные сплиты**: Если признак для основного сплита пропущен, используется «запасной» признак, который максимально коррелирует с основным (дает похожее разбиение выборки).
- **Отдельная категория**: Пропуск можно выделить в отдельное значение признака.

20.10 Регуляризация

- Pre-pruning (Критерии остановки):

- **Max depth**: Ограничивает глубину, предотвращая создание слишком специфичных правил.
- **Min samples split/leaf**: Запрещает создавать узлы, в которые попадает слишком мало объектов. Это защищает от подстройки под шум.
- **Min impurity decrease**: Сплит делается только если он значительно уменьшает ошибку.

- **Post-pruning (Стрижка)**: Позволяет дереву сначала «переобучиться» (построиться до конца), а затем удаляет узлы, которые не дают значимого прироста качества на валидации. Один из популярных методов — **Cost-Complexity Pruning** (минимизация функции $Loss + \alpha|Leaves|$).

20.11 Алгоритмические оптимизации (Трюки)

- Сортировка (Exact Greedy Algorithm):

- **Принцип**: Для каждого признака значения сортируются ($O(N \log N)$), после чего перебор всех возможных порогов занимает $O(N)$. Общая сложность поиска сплита: $O(D \cdot N \log N)$.
- **Плюсы**: Находит **идеальный порог**, обеспечивая максимальную точность для одного дерева.

- **Минусы/Сложность:** Огромное потребление памяти (нужно хранить отсортированные индексы для каждого признака). Плохо масштабируется на очень большие данные (миллионы строк).

- **Гистограммный метод (Histogram-based Algorithm):**

- **Принцип:** Значения признака разбиваются на k корзин (бинов, обычно $k = 256$). Вместо перебора всех значений N , перебираются только границы бинов.
- **Плюсы:**
 1. **Скорость:** Поиск сплита занимает $O(D \cdot k)$ вместо $O(D \cdot N)$.
 2. **Память:** Данные можно хранить в 'uint8' (номера бинов), что экономит до 4-8 раз больше места.
 3. **Subtraction Trick:** Гистограмму для большего ребенка можно получить вычитанием гистограммы меньшего ребенка из родительской ($O(k)$), что вдвое ускоряет расчеты.
- **Минусы:** Небольшая **потеря точности** из-за дискретизации (обычно нивелируется в ансамблях). Сложность реализации (нужно аккуратно обрабатывать границы и разреженные данные).

Antigravity: Гистограммный метод — это "секретный соус" современных библиотек (LightGBM, CatBoost), позволяющий обучать деревья на миллионах строк за секунды.

20.12 Подбор гиперпараметров: определение

В машинном обучении различают два типа переменных:

- **Параметры модели:** это веса, которые алгоритм находит самостоятельно в процессе обучения на данных (например, коэффициенты w в линейной регрессии).
- **Гиперпараметры:** это внешние настройки алгоритма, которые задаются исследователем до начала обучения. Они определяют саму структуру модели и способ её обучения (например, глубина дерева, количество соседей в KNN, сила регуляризации).

Подбор гиперпараметров — это процесс поиска таких значений, при которых модель показывает наилучшее качество на новых данных.

20.13 Ключевые гиперпараметры для популярных алгоритмов

Ниже приведен перечень основных гиперпараметров, которые чаще всего настраиваются для достижения лучшего качества моделей.

Линейные модели (Linear/Logistic Regression): • **C** или **alpha:** Коэффициент регуляризации. Чем меньше значение (для C), тем сильнее регуляризация.

- **penalty:** Тип штрафа (l1, l2, elasticnet).
- **solver:** Алгоритм оптимизации (liblinear, saga, lbfgs).

Метод k-ближайших соседей (k-NN): • **n_neighbors:** Число соседей k .

- **weights:** Весовая функция (uniform или distance).
- **p:** Параметр метрики Минковского (1 — Манхэттен, 2 — Евклид).

Деревья решений (Decision Trees): • **max_depth:** Максимальная глубина дерева (контроль переобучения).

- **min_samples_split:** Минимум объектов, необходимых для разделения узла.
- **min_samples_leaf:** Минимум объектов в листе.
- **criterion:** Критерий ветвления (gini, entropy).

Случайный лес (Random Forest): • **n_estimators:** Количество деревьев в лесу.

- **max_features:** Число признаков для поиска лучшего разбиения.
- *Плюс все параметры одиночного дерева.*

Градиентный бустинг (XGBoost, LightGBM, CatBoost): • **learning_rate:** Скорость обучения (шаг градиентного спуска).

- **n_estimators:** Количество итераций (деревьев).
- **max_depth:** Глубина деревьев.
- **subsample:** Доля выборки для обучения одного дерева.
- **reg_alpha / reg_lambda:** L1 и L2 регуляризация.

Метод опорных векторов (SVM): • **C:** Параметр штрафа за ошибки (баланс между отступом и точностью).

- **kernel:** Тип ядра (linear, rbf, poly).

- **gamma**: Коэффициент для нелинейных ядер.

Нейронные сети: • **learning_rate**: Скорость обучения.

- **batch_size**: Размер пакета данных.
- **optimizer**: Алгоритм оптимизации (Adam, SGD, RMSprop).
- **dropout_rate**: Коэффициент исключения нейронов (регуляризация).

20.14 Методология подбора

Чтобы оценка качества была честной, нельзя подбирать гиперпараметры на тех же данных, на которых модель обучается или на которых она окончательно тестируется.

20.14.1 Схема Train-Validation-Test

Классический подход подразумевает разделение всего датасета на три независимые части. Это необходимо, чтобы избежать «переобучения под тест»: если мы будем подбирать параметры, ориентируясь на тестовую выборку, мы получим слишком оптимистичную оценку, которая не подтвердится на реальных данных (эффект "подгонки" под тест).

- **Train (Обучающая)**: 60–80% данных. Используется исключительно для настройки внутренних весов модели (параметров).
- **Validation (Валидационная)**: 10–20% данных. Служит для оценки качества при разных гиперпараметрах. Именно по результатам на этой части мы выбираем лучший набор настроек.
- **Test (Тестовая)**: 10–20% данных. «Золотой стандарт» оценки. К этой части обращаются ровно один раз в самом конце.

Важное правило: если после оценки на Test вы решили «еще немного подкрутить» параметры, Test автоматически становится частью Validation, и вам нужны новые данные для честного финального теста.

Типичный процесс подбора:

1. Фиксируем конкретный набор гиперпараметров.
2. Обучаем модель на **Train**.
3. Замеряем метрику качества на **Validation**.
4. Повторяем шаги 1–3 для всех интересующих комбинаций.
5. Выбираем комбинацию с лучшим результатом на валидации.
6. (Опционально) Переобучаем финальную модель на объединенной выборке **Train + Validation**.
7. Делаем итоговый замер качества на **Test**.



Рис. 5: Схема разделения данных для честной оценки.



Рис. 6: Сравнение Grid Search и Random Search.

20.15 Кросс-валидация (k-Fold)

Если данных недостаточно для выделения отдельной валидационной выборки или мы хотим получить более устойчивую оценку качества, используется **k-Fold кросс-валидация**.

Принцип работы:

1. Обучающая выборка делится на k равных частей (фолдов).
 2. Модель обучается k раз. В каждой итерации один фолд используется как валидационный, а остальные $k - 1$ — как обучающие.
 3. Итоговая метрика — это среднее арифметическое результатов на всех k валидационных фолдах.
- **Stratified k-Fold:** Вариант для классификации, где в каждом фолде сохраняется то же процентное соотношение классов, что и во всей выборке. Это критично при сильном дисбалансе классов.
 - **Плюсы:** Более надежная оценка качества (низкая дисперсия метрики), использование всех данных и для обучения, и для валидации.
 - **Минусы:** Вычислительная сложность (модель нужно обучить k раз).

20.16 Grid Search (Поиск по сетке)

Принцип: Исследователь задает дискретный список значений для каждого гиперпараметра. Алгоритм строит декартово произведение этих списков и перебирает **абсолютно все возможные комбинации**.

- **Плюсы:**
 - Исчерпывающий поиск в заданных границах.
 - Простота интерпретации и воспроизводимость.

- **Минусы:**

- «Проклятие размерности»: количество комбинаций растет экспоненциально с добавлением новых параметров.
- Неэффективность: много времени тратится на перебор «неважных» параметров, которые слабо влияют на результат.

20.17 Random Search (Случайный поиск)

Принцип: Вместо фиксации сетки мы задаем распределения (например, равномерное или логарифмическое) для параметров. Значения выбираются случайно.

Почему это работает (согласно Bergstra & Bengio): В большинстве задач лишь небольшое количество гиперпараметров являются действительно важными. Grid Search тратит время на перебор неважных измерений. Random Search же исследует гораздо больше уникальных значений «важных» параметров за то же количество итераций.

- **Плюсы:**

- Значительно быстрее при большом количестве параметров.
- Не привязан к конкретному шагу сетки.
- Вероятностная гарантия: при $n = 60$ итерациях мы с вероятностью 95% найдем решение, входящее в топ-5% лучших в пространстве поиска.

- **Минусы:**

- Не гарантирует нахождение глобального максимума (может «промахнуться» мимо узкого пика).
- Результат зависит от случайности (нужен фиксированный `random_state`).

20.18 Продвинутые методы подбора

Когда Grid и Random Search становятся слишком дорогими, на помощь приходят адаптивные методы, которые «учатся» на результатах предыдущих итераций.

20.18.1 Байесовская оптимизация

Основная идея — не искать вслепую, а построить вероятностную модель зависимости целевой метрики от гиперпараметров.

- **Суррогатная модель (Surrogate Model):** Аппроксимирует реальную (дорогую) целевую функцию. Чаще всего используются *Гауссовские процессы* или *Random Forest*. Модель предсказывает не только среднее значение метрики в точке x , но и дисперсию (неопределенность) $\sigma(x)$.
- **Функция полезности (Acquisition Function):** Решает, какую точку исследовать следующей. Она балансирует между:
 - *Exploitation* (Эксплуатация): Проверка областей, где модель предсказывает высокий результат.
 - *Exploration* (Исследование): Проверка областей с высокой неопределенностью (где мы еще не были).

- **Популярные функции:** **EI** (Expected Improvement) — математическое ожидание улучшения, **UCB** (Upper Confidence Bound) — верхняя граница доверительного интервала.

20.18.2 Алгоритм TPE (Tree-structured Parzen Estimator)

Это «облегченный» и эффективный вариант байесовской оптимизации, используемый в Optuna и Hyperopt.

Интуиция: Вместо моделирования функции $P(y|x)$ (как в Гауссовских процессах), TPE моделирует плотность распределения параметров $P(x|y)$.

1. Все результаты делятся на две группы: «лучшие» (верхний квантиль) и «остальные».
2. Строятся два распределения: $l(x)$ для лучших и $g(x)$ для остальных.
3. Следующей выбирается точка, которая максимизирует отношение $l(x)/g(x)$ — то есть точка, которая с наибольшей вероятностью принадлежит к «хорошим» результатам и с наименьшей — к «плохим».

20.18.3 Population Based Training (PBT)

Метод, вдохновленный эволюцией, идеально подходит для тяжелых нейросетей. В отличие от других методов, PBT подбирает параметры **прямо в процессе обучения**, а не запускает обучение заново.

- **Exploit:** Слабые модели в популяции «умирают», копируя веса более успешных соседей.
- **Explore:** После копирования весов гиперпараметры модели-победителя слегка мутируют (добавляется шум), чтобы найти еще более удачную комбинацию.
- **Результат:** Мы получаем и обученную модель, и оптимальное расписание (schedule) изменения параметров (например, learning rate) за один проход.

20.18.4 Hyperband и Successive Halving

Эти алгоритмы фокусируются на эффективном распределении ресурсов (времени, данных).

- **Принцип:** Мы запускаем много конфигураций на очень малом количестве итераций.
- **Отсев:** После каждой стадии выживает только топ-1/ n лучших конфигураций, которым выделяется в n раз больше ресурсов.
- Это позволяет быстро отбросить явно плохие архитектуры и сосредоточиться на перспективных.

20.19 Опенсорс-библиотеки и инструменты

Выбор инструмента зависит от сложности модели и имеющихся вычислительных ресурсов.

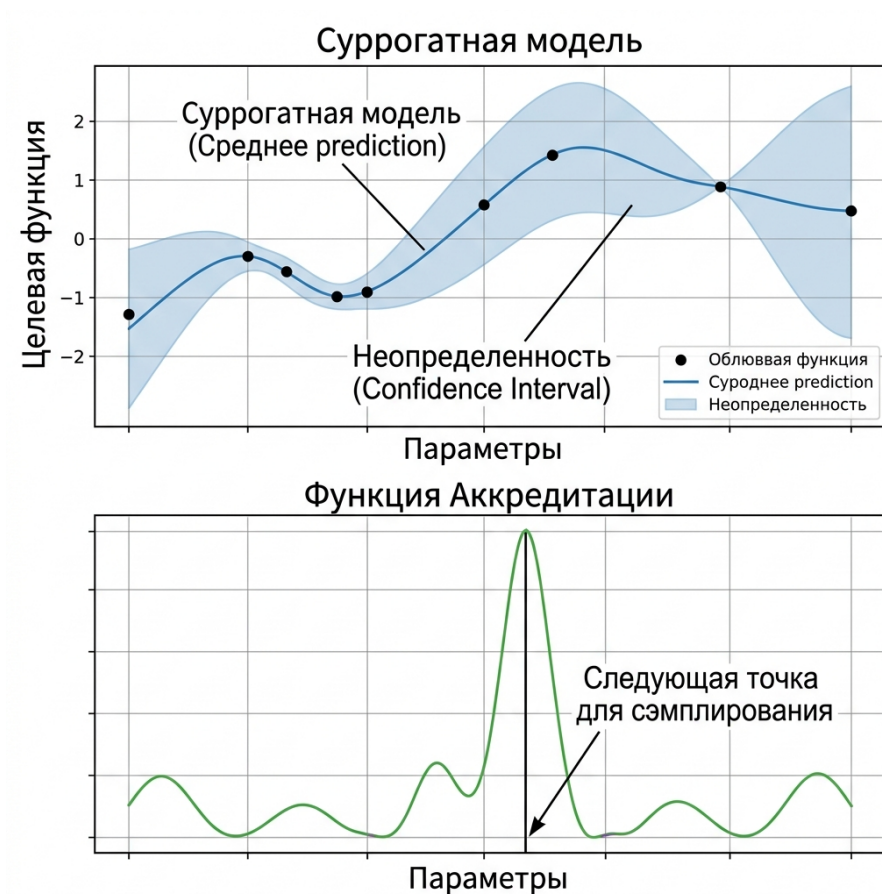


Рис. 7: Интуиция байесовской оптимизации.

Библиотека	Ключевые особенности и алгоритмы
Scikit-learn	Базовый стандарт. Предлагает <code>GridSearchCV</code> и <code>RandomizedSearchCV</code> . В новых версиях появился <code>HalvingGridSearchCV</code> , который использует метод последовательных сокращений (Successive Halving) для быстрой отбраковки плохих комбинаций.
Optuna	Лидер индустрии. Использует подход <i>Define-by-run</i> (пространство поиска описывается прямо внутри функции). • Pruning: Автоматически останавливает «безнадежные» итерации (trial), не дожидаясь конца обучения. • Алгоритмы: TPE, CMA-ES, Random Search. Отличная визуализация важности параметров.
Hyperopt	Классическая библиотека, популяризовавшая алгоритм TPE (Tree-structured Parzen Estimator). Имеет чуть более сложный синтаксис по сравнению с Optuna, но всё еще широко используется.

Scikit-Optimize	Специализируется на байесовской оптимизации . Главный плюс — наличие BayesSearchCV , который можно использовать как прямую замену стандартным методам из sklearn .
Ray Tune	Инструмент для масштабируемого подбора. Идеален для распределенных систем и тяжелых нейросетей. Поддерживает продвинутые стратегии, такие как PBT (Population Based Training) и Hyperband.
Keras Tuner	Специализированное решение для экосистемы TensorFlow/Keras. Позволяет подбирать не только параметры, но и саму архитектуру нейросети (количество слоев, фильтров и т.д.).

Antigravity: Начинай всегда с **Random Search**, чтобы быстро прощупать почву. Если задача критически важна — переходи на **Optuna**.

20.20 Что такое кросс-валидация?

Кросс-валидация (CV) — это статистический метод оценки обобщающей способности модели. Основная идея заключается в том, чтобы использовать имеющиеся данные как для обучения, так и для тестирования, но в разные моменты времени.

Зачем она нужна?

1. **Борьба с переобучением:** Мы проверяем модель на данных, которые она не видела в процессе настройки весов.
2. **Эффективное использование данных:** В отличие от простого Hold-out, где часть данных «замораживается», в CV каждый объект успевает побывать и в обучающей, и в тестовой выборке.
3. **Надежность оценки:** Усреднение результатов по нескольким разбиениям снижает влияние случайности (шума), который мог попасть в конкретный тестовый набор.

Кросс-валидация не обучает модель для продакшена. Она лишь дает нам честный ответ на вопрос: «Насколько хорош этот алгоритм и эти гиперпараметры?»

20.21 Метод Hold-out

Самый простой и быстрый способ. Мы делим данные на две части:

- **Train (обучающая):** на ней модель учится.
- **Test (тестовая/отложенная):** на ней мы измеряем качество.

Обычно используется пропорция **70/30** или **80/20**.

Минус: Оценка сильно зависит от того, *какие именно* объекты попали в тест. Если в тест случайно попали «легкие» объекты, метрика будет завышена.

20.22 Стратификация (Stratification)

Стратификация — это процесс формирования выборок таким образом, чтобы они сохраняли ключевые статистические характеристики всей исходной совокупности (обычно — распределение целевой переменной).

Почему это критично?

- **Дисбаланс классов:** В задачах классификации редкий класс может случайно не попасть в один из фолдов или, наоборот, переполнит его. Модель либо не увидит примеров для обучения, либо оценка качества будет искажена.
- **Регрессия:** При сильном разбросе таргета (например, цены на жилье с редкими «дворцами») важно, чтобы экстремальные значения распределялись равномерно. Для этого непрерывную переменную разбивают на бины (корзины) и стратифицируют по ним.

Результат: Использование стратификации снижает **дисперсию** (variance) оценки качества модели. Это значит, что результаты на разных фолдах будут более стабильными и репрезентативными.

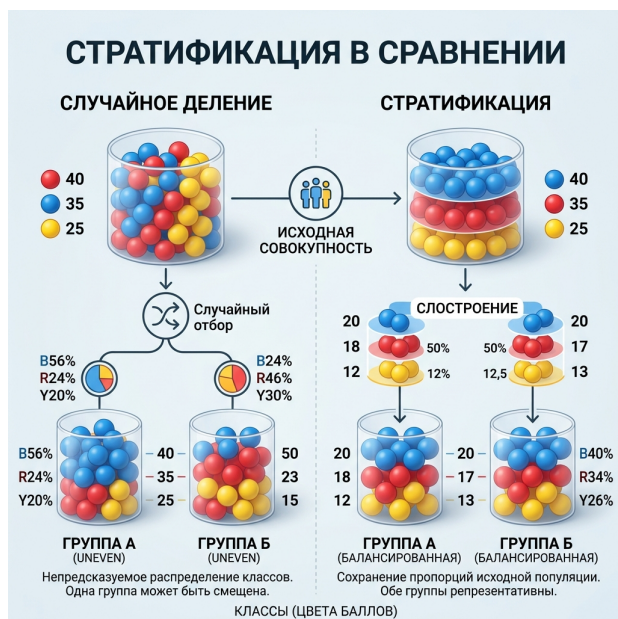


Рис. 8: Визуализация процесса стратификации.



Рис. 9: Схема k-Fold кросс-валидации.

20.23 k-Fold кросс-валидация

Это «золотой стандарт» оценки.

1. Весь датасет разбивается на k равных частей (фолдов), обычно $k = 5$ или $k = 10$.
2. Модель обучается k раз.
3. На каждой итерации один фолд становится тестовым, а остальные $k - 1$ — тренировочными.
4. Итоговая метрика — это **среднее арифметическое** результатов всех k итераций.

20.24 Leave-one-out (LOO)

Экстремальный случай k-Fold, где k равно количеству объектов в выборке (N).

- **Как это работает:** Мы обучаемся на всех объектах, кроме одного, и на этом одном тестируемся. И так N раз.
- **Плюс:** Мы используем максимум данных для обучения.
- **Минус:** Это невероятно долго и дорого в плане вычислений (если у вас миллион объектов, придется обучить модель миллион раз).

20.25 Stratified k-Fold

Это комбинация двух методов: данные разбиваются на k фолдов, но при этом в каждом фолде соблюдается **стратификация**. Это самый надежный способ оценки для задач классификации, особенно когда один класс встречается реже другого.

20.26 Кривые обучения (Learning Curves)

Кривые обучения — это мощный диагностический инструмент, который показывает зависимость ошибки (или метрики качества) от **размера обучающей выборки**. Они помогают ответить на главный вопрос: «Нужно ли нам больше данных или проблема в архитектуре модели?»

20.26.1 Как читать графики?

На одном графике строятся две кривые: ошибка на **трейне** и ошибка на **валидации**.

1. Ситуация: High Bias (Недообучение / Underfitting)

- **Признаки:** Обе кривые (трейн и валидация) сходятся к высокому значению ошибки. Зазор между ними минимален.
- **Вывод:** Модель слишком простая для этих данных.
- **Что делать:** Усложнить модель (добавить слои, степени признаков), добавить новые фичи. **Больше данных не поможет** — кривые уже вышли на «плато».

2. Ситуация: High Variance (Переобучение / Overfitting)

- **Признаки:** Ошибка на трейне очень низкая, а на валидации — высокая. Между кривыми наблюдается **большой зазор** (gap).
- **Вывод:** Модель «зазубрила» обучающую выборку, но не уловила общие закономерности.
- **Что делать:** Регуляризация, уменьшение количества признаков, упрощение модели. И, самое главное — **сбор дополнительных данных** поможет сузить зазор и улучшить обобщение.

3. Идеальный сценарий

- Обе кривые сходятся к низкому (приемлемому) значению ошибки с небольшим зазором между ними.

Кривые обучения позволяют сэкономить тысячи долларов на разметке данных, заранее показав, что добавление новых семплов уже не улучшит текущую архитектуру модели.

20.27 Когда стоит заподозрить, что оценка качества завышена?

Если вы видите метрику **0.999** или резкий скачок качества по сравнению с бейслайном, не спешите открывать шампанское. Скорее всего, вы столкнулись с иллюзией успеха.

Красные флаги (Red Flags):

1. **Слишком идеальные метрики:** Если в сложной реальной задаче (например, прогноз оттока клиентов) модель показывает точность $> 95\%$, это почти всегда признак утечки данных.
2. **Аномальная важность одного признака:** Если один признак имеет огромный вес в модели (Feature Importance), проверьте, не является ли он «ответом в другом обличье». *Пример:* Прогноз диагноза с использованием признака «ID назначенного лекарства».

3. **Разрыв между CV и Тестом:** Если на кросс-валидации всё идеально, а при первом же тесте на новых данных модель «рассыпается», значит, кросс-валидация была настроена неверно (например, не учтена временная структура или группировка данных).
4. **Метрика не соответствует бизнес-логике:** Модель отлично предсказывает редкое событие, но при этом путает самые очевидные случаи. Это часто указывает на переобучение под конкретные аномалии в обучающей выборке.

Antigravity: Лучший способ проверить модель на «честность» — это полностью отложить небольшой кусок данных (Hold-out set) в самом начале и не трогать его до самого финала. Если на нем результат сильно хуже, чем на CV — ищи утечку.

20.28 Примеры «подмешивания» тестовых данных (Data Leakage)

Это ситуация, когда информация из будущего или из теста просачивается в обучение. Модель «подсматривает» правильные ответы.



Рис. 10: Схематичное представление утечки данных.

Примеры:

- **Масштабирование (Scaling) до разбиения:** Вы посчитали среднее и дисперсию по *всему* датасету, а потом применили `StandardScaler`. Модель уже «знает» среднее значение тестовой выборки. **Правильно:** считать параметры только на трейне.
- **Временные ряды (Time Series):** Вы предсказываете курс акций сегодня, используя данные за завтра. Кросс-валидация моделей для такой задачи осложняется тем, что данные не должны пересекаться по времени: тренировочные данные должны идти до валидационных, а валидационных — до тестовых. С учётом этих особенностей фолды в кросс-валидации для временных рядов располагаются вдоль временной оси (например, метод *Time Series Split* или «скользящее окно»).
- **Дубликаты:** Если в данных есть дублирующие строки, один и тот же объект может попасть и в трейн, и в тест. Модель просто «вспомнит» ответ.

- **Косвенные признаки:** Например, предсказание вероятности рака, используя признак «прошел ли пациент химиотерапию». Терапия назначается *после* диагноза, поэтому она содержит ответ в себе.

Помни, кросс-валидация — это твой страховой полис. Лучше потратить время на обучение 5 фолдов сейчас, чем через месяц объяснять заказчику, почему «идеальная» модель на практике выдает случайные числа.

Antigravity: Всегда используй пайплайны. Это защитит тебя от 90% случаев утечки данных через предобработку.

21 Фундаментальные основы ансамблей

21.1 Дилемма смещения и разброса (Bias-Variance Tradeoff)

Любую ошибку модели можно разложить на три составляющие:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Bias (Смещение):** Математическое ожидание разности между истинным ответом и предсказанием. Высокое смещение означает, что модель слишком простая (**Недообучение**).
- **Variance (Разброс):** Вариативность предсказаний при смене обучающей выборки. Высокий разброс означает, что модель слишком сложная и подстраивается под шум (**Переобучение**).
- **Шум (Irreducible Error):** Ошибка, которую невозможно убрать (ошибки разметки, неполнота данных).



Рис. 11: Иллюстрация Bias и Variance на примере мишени.

21.2 Случайный лес (Random Forest)

Random Forest — это ансамбль глубоких решающих деревьев, обученных независимо друг от друга.

- **Как строится (Двойная случайность):**
 1. **Bagging (Bootstrap Aggregating):** Каждое дерево обучается на своей подвыборке данных, полученной с помощью бутстрапа (выборка с возвращением). Это дает деревьям «разный взгляд» на данные.
 2. **Метод случайных подпространств (Feature Subsampling):** В каждом узле при поиске лучшего сплита дерево выбирает не из всех D признаков, а из случайного подмножества размера $\sqrt{D}$. Это критически важно для **декорреляции** деревьев.

- **Зачем это нужно?:** Усреднение ответов N независимых моделей с одинаковым математическим ожиданием ошибки сохраняет **Bias**, но уменьшает **Variance** в N раз. Чем меньше корреляция между деревьями, тем эффективнее работает ансамбль.
- **Минусы:**
 - **Память:** Глубокие деревья занимают очень много места на диске.
 - **Скорость:** Работает медленнее бустинга на этапе предсказания (нужно прогнать объект через сотни глубоких деревьев).
 - **Экстраполяция:** Как и все деревья, не умеет предсказывать значения вне диапазона обучения.

22 Градиентный бустинг (GBDT)

22.1 Философское отличие: Бустинг vs Бэггинг

Чтобы понять бустинг, нужно вспомнить случайный лес (бэггинг).

- **Бэггинг** (Random Forest) строит много глубоких, умных деревьев **независимо** и усредняет их. Это борется с **разбросом** (variance).
- **Бустинг** строит цепочку из очень простых («глупых») деревьев **последовательно**. Каждое следующее дерево исправляет ошибки всех предыдущих. Это борется со **смещением** (bias).

Бустинг можно представить как гольфиста, цель которого — загнать мяч в лунку. Здесь положение мяча — ответ композиции. Каждый следующий удар — это та поправка, которую вносит очередной базовый алгоритм. Если гольфист всё делает правильно, то функция потерь будет уменьшаться с каждым ударом.

22.2 Математика: Градиентный спуск в функциональном пространстве

В GBDT мы не подбираем веса объектов или параметры разделяющей поверхности напрямую. Мы ищем саму **функцию** $f(x)$ в виде суммы базовых алгоритмов, минимизирующую функционал ошибки:

$$\mathcal{L}(f) = \sum_{i=1}^n L(y_i, f(x_i)) \rightarrow \min_f$$

22.2.1 Алгоритм по шагам

На шаге m у нас уже есть композиция $f_{m-1}(x) = \sum_{t=1}^{m-1} \eta h_t(x)$. Мы хотим найти такое дерево $h_m(x)$, чтобы:

$$f_m(x) = f_{m-1}(x) + \eta h_m(x)$$

где η — темп обучения (learning rate).

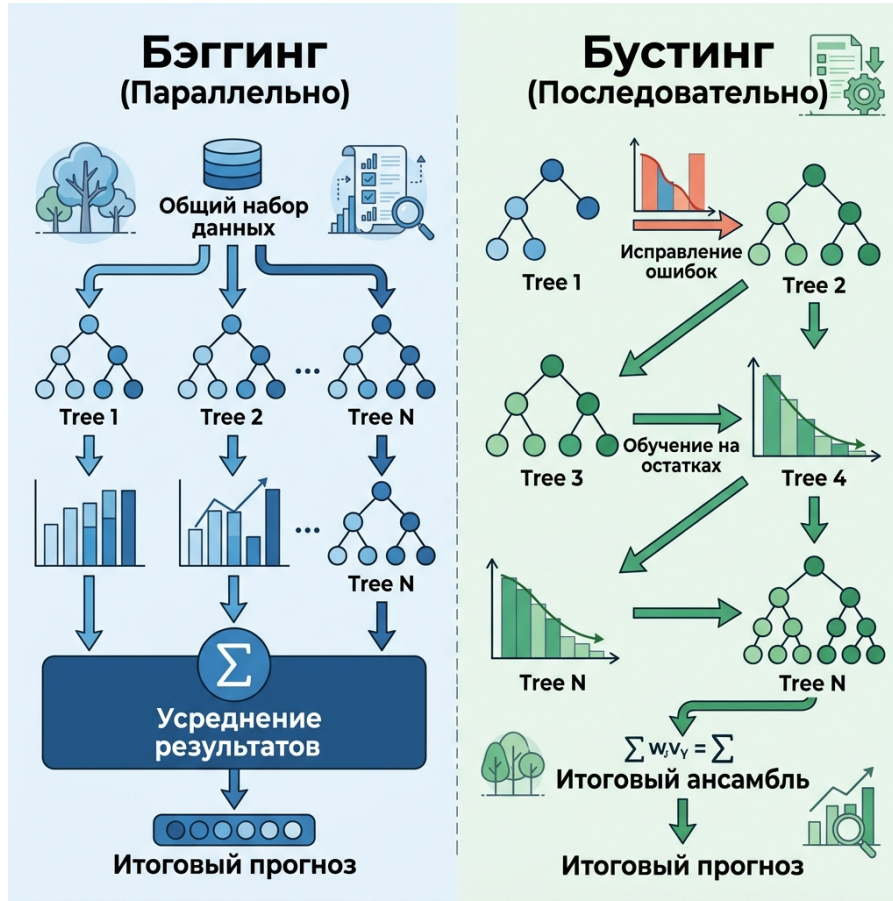


Рис. 12: Концептуальное различие между Бустингом и Бэггингом.

22.2.2 Почему это градиентный спуск?

Представим, что мы можем менять предсказания модели в каждой точке x_i независимо. Чтобы минимизировать $L(y_i, f(x_i))$, нам нужно сдвинуть $f(x_i)$ в сторону антиградиента:

$$f(x_i) \leftarrow f(x_i) - \underbrace{\eta \frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}}_{g_i}$$

Здесь g_i — это **градиент** функции потерь по предсказанию модели для i -го объекта.

22.2.3 Обучение на остатках

Так как мы ограничены семейством решающих деревьев, мы не можем сделать произвольный сдвиг. Поэтому мы обучаем новое дерево $h_m(x)$ предсказывать значения **антиградиента** $r_i = -g_i$:

$$h_m = \arg \min_h \sum_{i=1}^n (h(x_i) - r_i)^2$$

Для MSE антиградиент — это просто остатки $(y_i - f_{m-1}(x_i))$. Для других лоссов (например, LogLoss) антиградиент имеет более сложный вид, но логика остается прежней.

22.2.4 Сходимость

Метод сходится, потому что на каждой итерации мы добавляем в композицию функцию, которая максимально (в смысле L_2) приближает направление наискорейшего спуска в

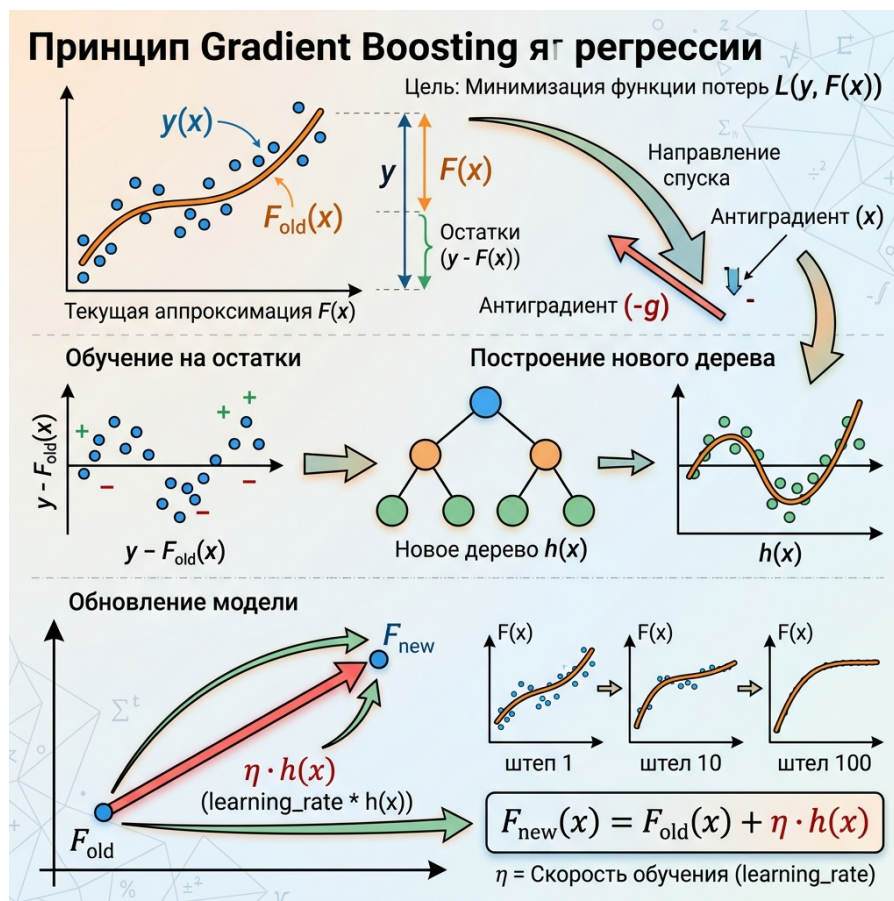


Рис. 13: Процесс обучения нового дерева на антиградиент функции потерь.

пространстве ответов. Таким образом, с каждым шагом общая ошибка на обучающей выборке гарантированно уменьшается (при достаточно малом η).

22.3 Ограничение сложности и Регуляризация

Бустинг по своей природе — очень жадный алгоритм. Если его не ограничивать, он мгновенно «зазубрит» обучающую выборку (переобучится). Для борьбы с этим используются два рычага: ограничение базовых моделей и внешняя регуляризация.

22.3.1 Почему деревья должны быть неглубокими?

В отличие от случайного леса, где деревья должны быть глубокими, в GBDT используются «слабые» ученики (обычно глубиной **3–6 уровней**).

- **Смещение vs Разброс:** Бустинг борется со смещением. Если брать глубокие деревья (у которых разброс и так высокий), ансамбль станет крайне нестабильным.
- **Принцип композиции:** Сила бустинга в *комбинации* тысяч простых правил. Каждое дерево должно вносить лишь небольшое уточнение, исправляя грубые ошибки предыдущих.
- **Взаимодействия:** Дерево глубины d способно уловить взаимодействие (interaction) максимум d признаков. Практика показывает, что взаимодействий 3–6 порядка достаточно для большинства задач.

22.3.2 Методы борьбы с переобучением (Регуляризация)

1. **Learning Rate (Shrinkage)**: Мы умножаем предсказание каждого нового дерева на коэффициент $\eta \ll 1$ (например, 0.01). Это заставляет алгоритм «идти к минимуму маленькими шагами», что делает его устойчивым к шуму. Обучение при этом требует больше итераций, но качество на выходе выше.
2. **Стохастический бустинг (Subsampling)**: Каждое дерево обучается не на всей выборке, а на ее случайном подмножестве (по строкам) или на случайном наборе признаков (по столбцам). Это вносит элемент случайности и мешает деревьям слишком сильно коррелировать между собой.
3. **Веса листьев (L2 regularization)**: В итоговую функцию потерь добавляется штраф за слишком большие значения в листьях (параметр λ в XGBoost). Это делает предсказания деревьев более консервативными.
4. **Early Stopping (Ранняя остановка)**: Мы прекращаем добавлять новые деревья, когда ошибка на **валидационной** выборке перестает падать. Это самый эффективный способ найти оптимальное количество итераций M .

23 Тройка лидеров: XGBoost, LightGBM, CatBoost

Каждая из этих библиотек — это эволюция классического GBDT, решающая проблемы скорости, памяти и качества.

23.1 XGBoost (eXtreme Gradient Boosting)

Первый массовый лидер, который принес бустингу популярность на Kaggle.

- **Как строится**: Использует **Level-wise** рост (дерево растет по уровням, сохраняя баланс).
- **Математическая фишка**: Аппроксимация функции потерь рядом Тейлора до **второго порядка** (использует и градиент, и **гессиан** — вторую производную). Это позволяет точнее находить направление спуска.
- **Плюсы**:
 - **Sparsity-aware**: Умеет эффективно обрабатывать пропуски (направляет их в «лучшую» сторону при сплите).
 - **Регуляризация**: Встроенные L1 и L2 штрафы прямо в функции поиска сплита.

23.2 LightGBM (от Microsoft)

Создан для работы с **Big Data**. Когда данных миллионы, XGBoost становится слишком медленным.

- **Как строится**: **Leaf-wise** рост (разбивает тот лист, который дает максимальное падение лосса, независимо от уровня). Деревья могут быть очень глубокими и асимметричными.
- **Математические фишки**:

- **GOSS**: Отбрасывает объекты с маленькими градиентами (на них модель уже выучилась) и оставляет только «сложные».
- **EFB**: Объединяет взаимоисключающие признаки в один (актуально для разреженных данных).

- **Плюсы**: Экстремальная скорость и низкое потребление памяти.

23.3 CatBoost (от Яндекса)

Король табличных данных с большим количеством категориальных признаков.

- **Как строится: Symmetric Trees (Oblivious Trees)**. На каждом уровне дерева используется один и тот же признак и один и тот же порог для всех узлов. Это работает как очень сильная регуляризация.
- **Математические фишки**:
 - **Ordered Boosting**: Решает проблему «смещения предсказания» (prediction shift), обучая модели на перестановках данных.
 - **Native Categorical Support**: Использует продвинутую версию Target Encoding, которая не приводит к утечке данных.
- **Плюсы**: Минимум тюнинга «из коробки», лучшая точность на категориях, очень быстрый инференс (из-за симметрии деревьев).

Параметр	XGBoost	LightGBM	CatBoost
Рост дерева	Level-wise	Leaf-wise	Symmetric
Скорость	Средняя	Высокая	Средняя (на обучении)
Категории	One-Hot / Числа	Индексация	Native (Best)
Пропуски	Авто-направление	Авто-направление	Режим "Min/Max"

Таблица 5: Сравнение ключевых характеристик библиотек бустинга.

23.4 Интерпретация: Feature Importance

Понимание того, почему модель приняла решение, не менее важно, чем сама точность. В бустинге есть три основных способа оценить важность признаков.

1. Weight / Frequency (Частота)

- **Суть**: Считает, сколько раз данный признак был выбран для разделения (сплита) во всех деревьях ансамбля.
- **Минусы**: Очень **ненадежный** метод. Он поощряет непрерывные признаки с большим количеством уникальных значений (например, ID или случайный шум), так как дерево может «пилить» их бесконечное число раз. При этом реальный вклад в точность может быть нулевым.

2. Gain (Прирост)

- **Суть**: Суммирует вклад каждого признака в уменьшение функции потерь (Gain) во всех узлах, где он использовался.

- **Плюсы:** Самый популярный встроенный метод. Он показывает, насколько реально «помог» признак снизить ошибку.
- **Минусы:** Всё еще может быть смещен в сторону признаков с высокой кардинальностью (много категорий).

3. Permutation Importance (Важность через перемешивание)

- **Суть:** Мы берем отложенную выборку, замеряем качество. Затем перемешиваем значения одного признака (разрушая его связь с таргетом) и смотрим, насколько упала метрика. Чем сильнее падение — тем важнее признак.
- **Плюсы: Самый честный** метод. Он не зависит от структуры дерева и показывает реальную ценность признака для предсказательной силы на новых данных.
- **Минусы:** Требуется много времени на вычисления (нужно прогонять предсказания много раз). Плохо работает, если признаки сильно коррелируют между собой.

Antigravity: На практике всегда смотри на **Gain**, но проверяй его через **Permutation Importance** или **SHAP values**. Если признак имеет высокий **Gain**, но нулевой **Permutation** — скорее всего, модель нашла в нем шум или «чит», который не работает на новых данных.

*Главное при работе с бустингом — следить за графиком лосса на валидации. Как только ошибка на валидации перестала падать, а на трейне продолжает — немедленно останавливайся (это называется *Early Stopping*).*

Antigravity: Для большинства табличных задач CatBoost — это выбор по умолчанию благодаря отличной работе с категориями и регуляризации.